MSPA - 2

NAME :- DEVANSHU PARSHOINIKAR
BRANCH :- COMPUTER TECHNOLOGY
ROLL NO. :- 49

Problem statement: Develop a simple Student Task Manager that allows students to add tasks, mark tasks as completed, delete tasks, and view pending tasks.

Code:-

Index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Student Task Manager</title>

  <link rel="stylesheet" type="" href="style.css">

</head>
<body>
  <div class="container">
    <header>
      <div class="brand">
        <h1>📚 Student Task Manager</h1>
        <p>Stay on top of your studies, assignments, and exams!</p>
      </div>
    </header>

    <div class="stats-container">
      <div class="stat-card">
        <h3>Total Tasks</h3>
        <div class="number" id="stat-total">0</div>
      </div>
      <div class="stat-card pending">
        <h3>Pending</h3>
        <div class="number" id="stat-pending">0</div>
      </div>
      <div class="stat-card completed">
        <h3>Completed</h3>
        <div class="number" id="stat-completed">0</div>
      </div>
    </div>

    <div class="card">
      <h2>✨ Add New Task</h2>
      <form id="task-form">
```

```
        <div class="form-grid">
          <div class="form-group full-width">
            <label for="task-title">Task Title *</label>
            <input type="text" id="task-title" placeholder="e.g., Physics Chapter 4 Problem Set" required>
          </div>
          <div class="form-group full-width">
            <label for="task-desc">Description / Notes</label>
            <textarea id="task-desc" placeholder="Add any details, reading pages, or links..."></textarea>
          </div>
          <div class="form-group">
            <label for="task-date">Due Date</label>
            <input type="date" id="task-date">
          </div>
          <div class="form-group">
            <label for="task-priority">Priority Level</label>
            <select id="task-priority">
              <option value="Low">Low Priority</option>
              <option value="Medium" selected>Medium Priority</option>
              <option value="High">High Priority</option>
            </select>
          </div>
        </div>
        <button type="submit" class="btn btn-primary" style="margin-top: 0.5rem; width: 100%;">
          <span>➕ Add Task</span>
        </button>
      </form>
    </div>

    <div class="controls-bar">
      <div class="search-box">
        <span class="search-icon">🔍</span>
        <input type="text" id="search-input" placeholder="Search tasks by title...">
      </div>
      <div class="filter-tabs">
        <button class="tab-btn active" data-filter="all">All</button>
        <button class="tab-btn" data-filter="pending">Pending</button>
        <button class="tab-btn" data-filter="completed">Completed</button>
      </div>
    </div>

    <div class="task-list" id="task-list">
    </div>
  </div>

  <script src="app.js">
  </script>
</body>
</html>
```

Style.css

```
* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

body {
  background-color: #f8fafc;
  color: #1e293b;
  min-height: 100vh;
  display: flex;
  flex-direction: column;
  align-items: center;
  padding: 2rem 1rem;
}

.container {
  width: 100%;
  max-width: 850px;
  margin: 0 auto;
}

header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin-bottom: 2rem;
  flex-wrap: wrap;
  gap: 1rem;
}

.brand h1 {
  font-size: 1.85rem;
  font-weight: 800;
  color: #1e293b;
  display: flex;
  align-items: center;
  gap: 0.5rem;
}

.brand p {
  font-size: 0.9rem;
  color: #64748b;
  margin-top: 0.2rem;
}

.stats-container {
  display: grid;
  grid-template-columns: repeat(3, minmax(100px, 1fr));
  gap: 1rem;
  margin-bottom: 2rem;
}
```

```
.stat-card {
  background: #ffffff;
  padding: 1.25rem;
  border-radius: 16px;
  box-shadow: 0 1px 3px rgba(0,0,0,0.05), 0 1px 2px rgba(0,0,0,0.1);
  border: 1px solid #e2e8f0;
  text-align: center;
  transition: all 0.25s cubic-bezier(0.4, 0, 0.2, 1);
}

.stat-card:hover {
  transform: translateY(-2px);
  box-shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.05), 0 2px 4px -1px rgba(0, 0, 0, 0.03);
}

.stat-card h3 {
  font-size: 0.85rem;
  text-transform: uppercase;
  letter-spacing: 0.05em;
  color: #64748b;
  margin-bottom: 0.4rem;
}

.stat-card .number {
  font-size: 1.75rem;
  font-weight: 700;
  color: #4f46e5;
}

.stat-card.pending .number { color: #f59e0b; }
.stat-card.completed .number { color: #10b981; }

.card {
  background: #ffffff;
  border-radius: 16px;
  padding: 1.75rem;
  box-shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.05), 0 2px 4px -1px rgba(0, 0, 0, 0.03);
  border: 1px solid #e2e8f0;
  margin-bottom: 2rem;
}

.card h2 {
  font-size: 1.2rem;
  font-weight: 700;
  margin-bottom: 1.25rem;
  display: flex;
  align-items: center;
  gap: 0.5rem;
}

.form-grid {
  display: grid;
  grid-template-columns: 1fr 1fr;
```

```
  gap: 1rem;
}

.form-group {
  display: flex;
  flex-direction: column;
  gap: 0.4rem;
  margin-bottom: 1rem;
}

.form-group.full-width {
  grid-column: span 2;
}

label {
  font-size: 0.85rem;
  font-weight: 600;
  color: #1e293b;
}

input, textarea, select {
  width: 100%;
  padding: 0.75rem 1rem;
  border: 1px solid #e2e8f0;
  border-radius: 10px;
  font-size: 0.95rem;
  font-family: inherit;
  color: #1e293b;
  background-color: #fff;
  transition: all 0.25s cubic-bezier(0.4, 0, 0.2, 1);
}

input:focus, textarea:focus, select:focus {
  outline: none;
  border-color: #4f46e5;
  box-shadow: 0 0 0 3px #e0e7ff;
}

textarea {
  resize: vertical;
  min-height: 80px;
}

.btn {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  gap: 0.5rem;
  padding: 0.75rem 1.5rem;
  font-size: 0.95rem;
  font-weight: 600;
  border-radius: 10px;
  border: none;
  cursor: pointer;
```

```
    transition: all 0.25s cubic-bezier(0.4, 0, 0.2, 1);
    font-family: inherit;
}

.btn-primary {
    background-color: #4f46e5;
    color: white;
    box-shadow: 0 2px 4px rgba(79, 70, 229, 0.2);
}

.btn-primary:hover {
    background-color: #4338ca;
    transform: translateY(-1px);
}

.controls-bar {
    display: flex;
    justify-content: space-between;
    align-items: center;
    margin-bottom: 1.5rem;
    gap: 1rem;
    flex-wrap: wrap;
}

.search-box {
    position: relative;
    flex: 1;
    min-width: 240px;
}

.search-box input {
    padding-left: 2.5rem;
}

.search-icon {
    position: absolute;
    left: 0.85rem;
    top: 50%;
    transform: translateY(-50%);
    color: #64748b;
}

.filter-tabs {
    display: flex;
    background: #e2e8f0;
    padding: 0.25rem;
    border-radius: 10px;
    gap: 0.25rem;
}

.tab-btn {
    background: transparent;
    border: none;
    padding: 0.5rem 1rem;
```

```
  font-size: 0.85rem;
  font-weight: 600;
  color: #64748b;
  border-radius: 6px;
  cursor: pointer;
  transition: all 0.25s cubic-bezier(0.4, 0, 0.2, 1);
  font-family: inherit;
}

.tab-btn.active {
  background: #ffffff;
  color: #4f46e5;
  box-shadow: 0 1px 3px rgba(0,0,0,0.05), 0 1px 2px rgba(0,0,0,0.1);
}

.task-list {
  display: flex;
  flex-direction: column;
  gap: 1rem;
}

.task-item {
  background: #ffffff;
  border-radius: 16px;
  padding: 1.25rem;
  box-shadow: 0 1px 3px rgba(0,0,0,0.05), 0 1px 2px rgba(0,0,0,0.1);
  border: 1px solid #e2e8f0;
  display: flex;
  align-items: flex-start;
  gap: 1rem;
  transition: all 0.25s cubic-bezier(0.4, 0, 0.2, 1);
  position: relative;
  overflow: hidden;
}

.task-item:hover {
  box-shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.05), 0 2px 4px -1px rgba(0, 0, 0, 0.03);
  transform: translateY(-1px);
}

.task-item.completed {
  background-color: #f1f5f9;
  opacity: 0.85;
}

.task-item.completed .task-title {
  text-decoration: line-through;
  color: #64748b;
}

.task-item.completed .task-desc {
  color: #64748b;
}
```

```
.task-checkbox {
  margin-top: 0.2rem;
  width: 1.25rem;
  height: 1.25rem;
  accent-color: #10b981;
  cursor: pointer;
}

.task-content {
  flex: 1;
}

.task-header-row {
  display: flex;
  align-items: center;
  gap: 0.75rem;
  margin-bottom: 0.3rem;
  flex-wrap: wrap;
}

.task-title {
  font-size: 1.05rem;
  font-weight: 700;
  color: #1e293b;
}

.badge {
  font-size: 0.7rem;
  font-weight: 700;
  padding: 0.2rem 0.6rem;
  border-radius: 20px;
  text-transform: uppercase;
  letter-spacing: 0.03em;
}

.badge-low { background-color: #d1fae5; color: #065f46; }
.badge-medium { background-color: #fef3c7; color: #92400e; }
.badge-high { background-color: #fee2e2; color: #991b1b; }

.task-desc {
  font-size: 0.9rem;
  color: #64748b;
  margin-bottom: 0.75rem;
  line-height: 1.4;
  white-space: pre-line;
}

.task-footer {
  display: flex;
  align-items: center;
  gap: 1rem;
  font-size: 0.8rem;
  color: #64748b;
}
```

```
.task-due {
  display: flex;
  align-items: center;
  gap: 0.3rem;
}

.task-due.overdue {
  color: #ef4444;
  font-weight: 600;
}

.btn-icon {
  background: transparent;
  border: none;
  color: #64748b;
  cursor: pointer;
  padding: 0.4rem;
  border-radius: 6px;
  transition: all 0.25s cubic-bezier(0.4, 0, 0.2, 1);
  display: flex;
  align-items: center;
  justify-content: center;
}

.btn-icon:hover {
  background-color: #fee2e2;
  color: #ef4444;
}

.empty-state {
  text-align: center;
  padding: 3rem 1rem;
  background: #ffffff;
  border-radius: 16px;
  border: 1px dashed #e2e8f0;
  color: #64748b;
}

.empty-state span {
  font-size: 2.5rem;
  display: block;
  margin-bottom: 0.5rem;
}
```

App.js

```
let tasks = JSON.parse(localStorage.getItem('student_tasks')) || [];
let currentFilter = 'all';
let searchQuery = '';

const taskForm = document.getElementById('task-form');
const taskTitleInput = document.getElementById('task-title');
const taskDescInput = document.getElementById('task-desc');
const taskDateInput = document.getElementById('task-date');
const taskPriorityInput = document.getElementById('task-priority');
const taskListContainer = document.getElementById('task-list');
const searchInput = document.getElementById('search-input');
const filterTabs = document.querySelectorAll('.tab-btn');

const statTotal = document.getElementById('stat-total');
const statPending = document.getElementById('stat-pending');
const statCompleted = document.getElementById('stat-completed');

function saveTasks() {
  localStorage.setItem('student_tasks', JSON.stringify(tasks));
}

function updateStats() {
  const total = tasks.length;
  const completed = tasks.filter(t => t.completed).length;
  const pending = total - completed;

  statTotal.textContent = total;
  statPending.textContent = pending;
  statCompleted.textContent = completed;
}

function checkDueDate(dateString) {
  if (!dateString) return { text: '', isOverdue: false };
  const today = new Date();
  today.setHours(0, 0, 0, 0);
  const dueDate = new Date(dateString);
  dueDate.setHours(0, 0, 0, 0);
  const diffTime = dueDate - today;
  const diffDays = Math.ceil(diffTime / (1000 * 60 * 60 * 24));
  let text = `Due: ${dateString}`;
  let isOverdue = false;
  if (diffDays < 0) {
    text = `Overdue by ${Math.abs(diffDays)} day(s)`;
    isOverdue = true;
  } else if (diffDays === 0) {
    text = `Due Today!`;
    isOverdue = true;
  } else if (diffDays === 1) {
    text = `Due Tomorrow`;
  }
```

```
  return { text, isOverdue };
}

function renderTasks() {
  const filteredTasks = tasks.filter(task => {
    const matchesSearch = task.title.toLowerCase().includes(searchQuery.toLowerCase());
    if (currentFilter === 'pending') return matchesSearch && !task.completed;
    if (currentFilter === 'completed') return matchesSearch && task.completed;
    return matchesSearch;
  });

  if (filteredTasks.length === 0) {
    taskListContainer.innerHTML = `
          <div class="empty-state">
            <span>  </span>
            <p>No tasks found. Time to relax or add a new assignment!</p>
          </div>
        `;
    updateStats();
    return;
  }

  taskListContainer.innerHTML = filteredTasks.map(task => {
    const dueInfo = checkDueDate(task.dueDate);
    const priorityClass = `badge-${task.priority.toLowerCase()}`;

    return `
          <div class="task-item ${task.completed ? 'completed' : ''}" data-id="${task.id}">
            <input type="checkbox" class="task-checkbox" ${task.completed ? 'checked' : ''}
onchange="toggleTask(${task.id})">
            <div class="task-content">
              <div class="task-header-row">
                <span class="task-title">${escapeHTML(task.title)}</span>
                <span class="badge ${priorityClass}">${task.priority}</span>
              </div>
              ${task.desc ? `<div class="task-desc">${escapeHTML(task.desc)}</div>` : ''}
              <div class="task-footer">
                ${task.dueDate ? `
                  <div class="task-due ${dueInfo.isOverdue && !task.completed ? 'overdue' : ''}">
                     ${dueInfo.text}
                  </div>
                ` : ''}
              </div>
            </div>
            <button class="btn-icon" onclick="deleteTask(${task.id})" title="Delete Task">
              
            </button>
          </div>
        `;
  }).join('');

  updateStats();
```

```
}

function escapeHTML(str) {
  return str.replace(/[&<>'"]/g,
    tag => ({ '&': '&amp;', '<': '&lt;', '>': '&gt;', "'": '&#39;', '"': '&quot;' }[tag] || tag)
  );
}

taskForm.addEventListener('submit', (e) => {
  e.preventDefault();

  const title = taskTitleInput.value.trim();
  const desc = taskDescInput.value.trim();
  const dueDate = taskDateInput.value;
  const priority = taskPriorityInput.value;

  if (!title) return;

  const newTask = {
    id: Date.now(),
    title,
    desc,
    dueDate,
    priority,
    completed: false
  };

  tasks.unshift(newTask);
  saveTasks();
  renderTasks();

  taskForm.reset();
  taskPriorityInput.value = 'Medium';
});

window.toggleTask = function(id) {
  tasks = tasks.map(task => {
    if (task.id === id) {
      return { ...task, completed: !task.completed };
    }
    return task;
  });
  saveTasks();
  renderTasks();
};

window.deleteTask = function(id) {
  tasks = tasks.filter(task => task.id !== id);
  saveTasks();
  renderTasks();
};

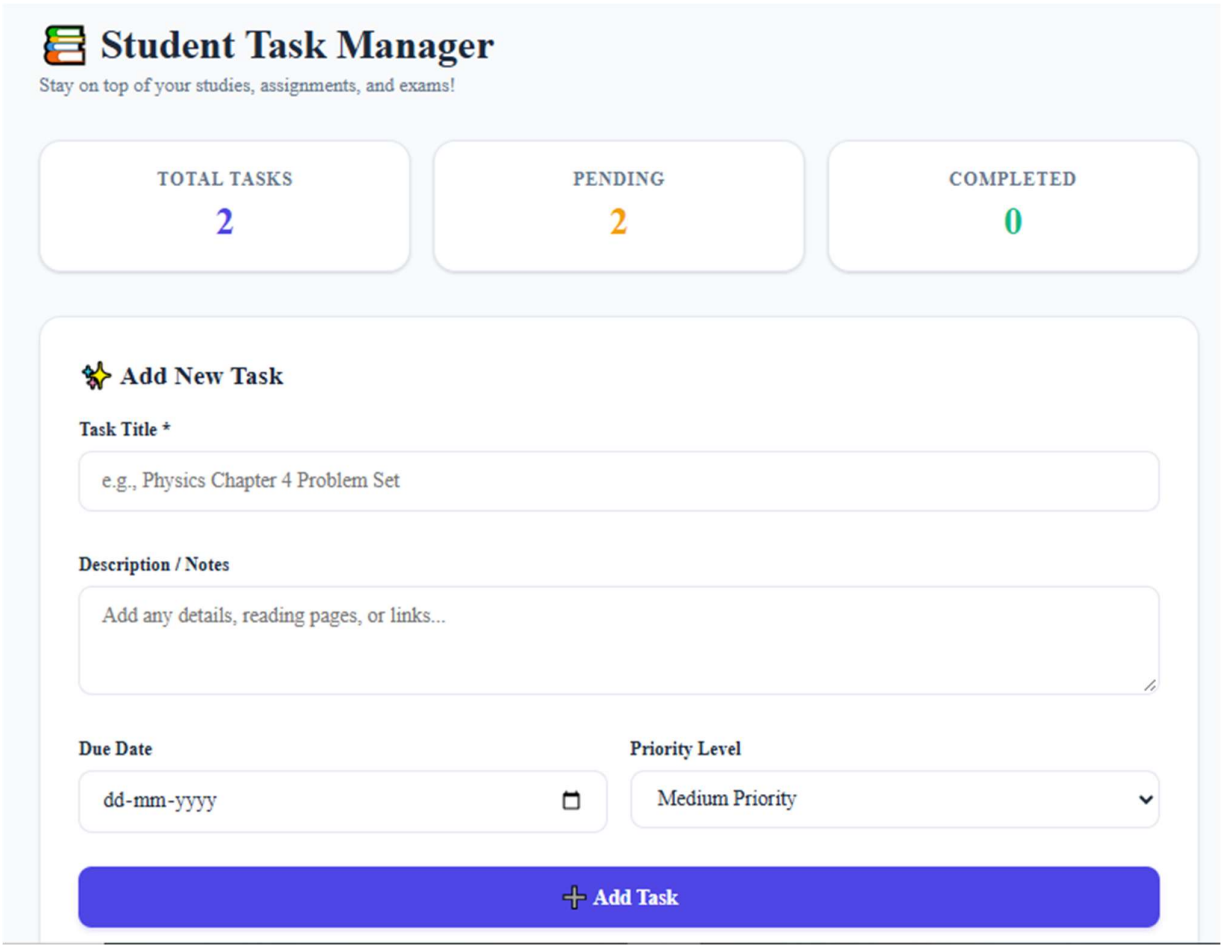
searchInput.addEventListener('input', (e) => {
  searchQuery = e.target.value;
```

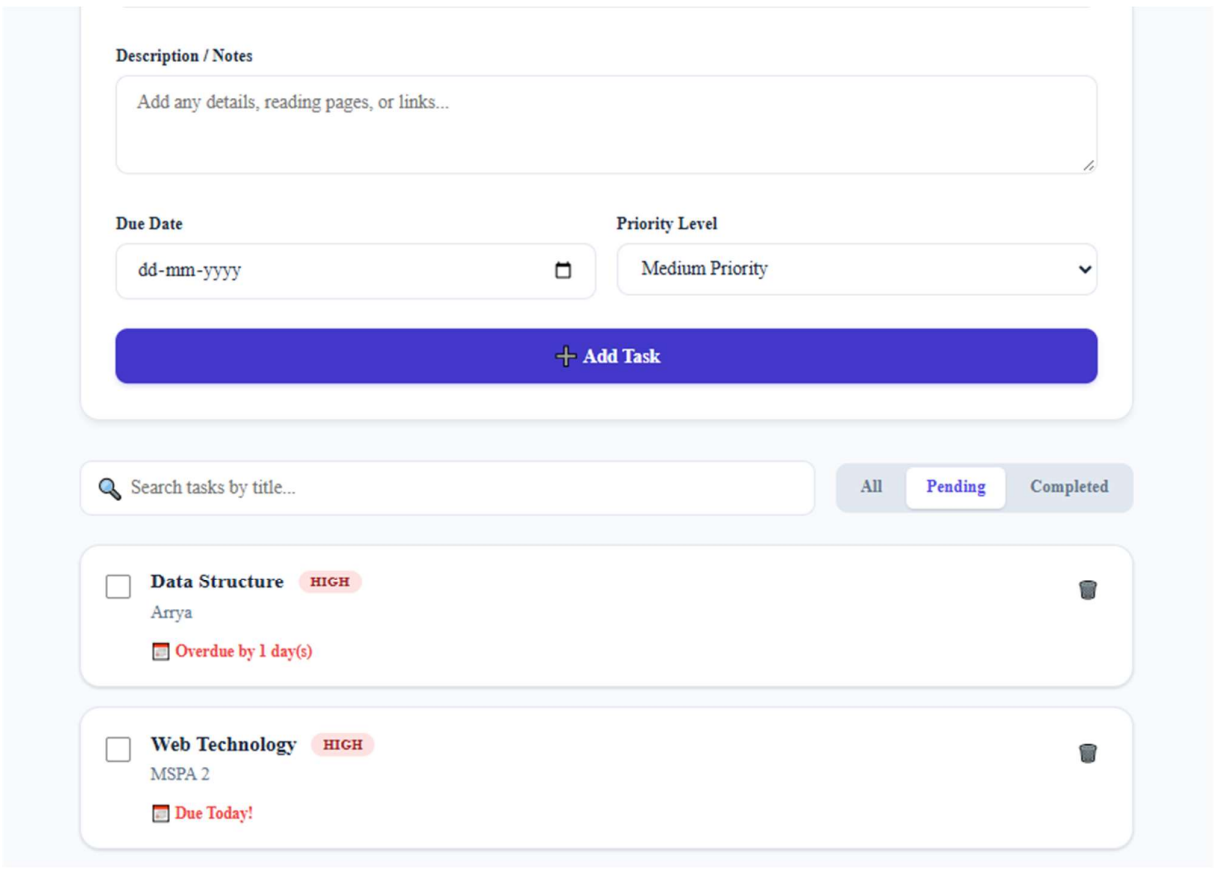
```
  renderTasks();
});

filterTabs.forEach(tab => {
  tab.addEventListener('click', (e) => {
    filterTabs.forEach(t => t.classList.remove('active'));
    e.target.classList.add('active');
    currentFilter = e.target.getAttribute('data-filter');
    renderTasks();
  });
});

renderTasks();
```

Output :-


Student Task Manager
Stay on top of your studies, assignments, and exams!
TOTAL TASKS
2
PENDING
2
COMPLETED
0
Add New Task
Task Title *
e.g., Physics Chapter 4 Problem Set
Description / Notes
Add any details, reading pages, or links...
Due Date
dd-mm-yyyy
Priority Level
Medium Priority
Add Task


Description / Notes
Add any details, reading pages, or links...
Due Date
dd-mm-yyyy
Priority Level
Medium Priority
Add Task
Search tasks by title...
All
Pending
Completed
Data Structure
HIGH
Arrya
Overdue by 1 day(s)
Web Technology
HIGH
MSPA 2
Due Today!